

Entity Relationship Data Model

Entity-Relationship Data Model

The **Entity-Relationship (ER) Data Model** is a **conceptual data model** used to describe the data that an organization needs to store and the relationships among that data. It is commonly used during the **database design process**, before the data is converted into relational tables.

An **Entity-Relationship Diagram (ERD)** is the **graphical representation of an Entity-Relationship Data Model**.

The five main components of an ERD:

Component	Purpose	Chen notation
Entity	A person, place, object, event, or concept about which data is stored	Rectangle
Attribute	A property or characteristic of an entity	Oval
Relationship	Describes how entities are associated	Diamond
Cardinality	Specifies the maximum number of entity instances that can participate in a relationship	1:1, 1:N, M:N
Participation	Specifies whether an entity's participation in a relationship is optional or mandatory	Single line = partial; double line = total

There are also variations within these components

For **entities**, you can have:

- Strong entity

- Weak entity

For **attributes**, you can have:

- Simple attribute

- Composite attribute

- Key attribute

- Multivalued attribute

- Derived attribute

- Partial key (for weak entities)

For **relationships**, you can have:

- Unary (recursive)

- Binary

- Ternary

- Identifying relationship (for weak entities)

Different **notations/styles** used to draw **Entity-Relationship Diagrams**, the most common are:

ERD Notation	Main Characteristics	Common Use
Chen Notation	Rectangles for entities, ovals for attributes, diamonds for relationships	Teaching and conceptual database design
Crow's Foot Notation	Entities shown as table-like boxes; relationship lines use "crow's feet" to indicate many	Very common in practical database design
UML Class Diagram	Classes/entities shown as boxes with attributes; multiplicities such as 1, 0..*	Software engineering and object-oriented design
IDEF1X	Detailed notation emphasizing primary keys, foreign keys, and identifying relationships	Enterprise and technical database modeling
Barker Notation	Similar to Crow's Foot; uses lines and symbols for optionality and cardinality	Historically associated with Oracle modeling

Chen and Crow's Foot notation can both represent the fundamental structure of an ER model, but they emphasize different aspects of database design.

Chen notation is particularly suited to conceptual modeling, while Crow's Foot notation is commonly used for logical database modeling and shows cardinality and optionality very clearly.

Chen Notation ERD (Entity-Relationship Diagram) is a graphical method used to **design and represent the structure of a database**.

It was introduced by computer scientist **Peter Chen in 1976**. Chen notation shows the **entities** in a database, their **attributes**, and the **relationships** between those entities.



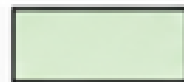
Entities

An **entity** represents a person, place, object, event, or concept about which the database needs to store information.

Examples include:

- Employee
- Department
- Student
- Course
- Customer
- Product

In **Chen notation**, an entity is represented by a **rectangle**.

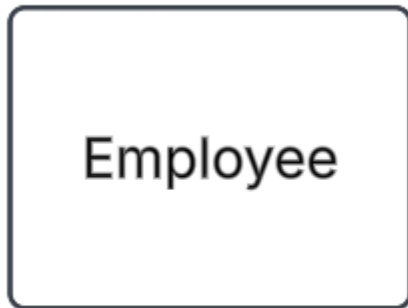


= Entity



= Weak Entity

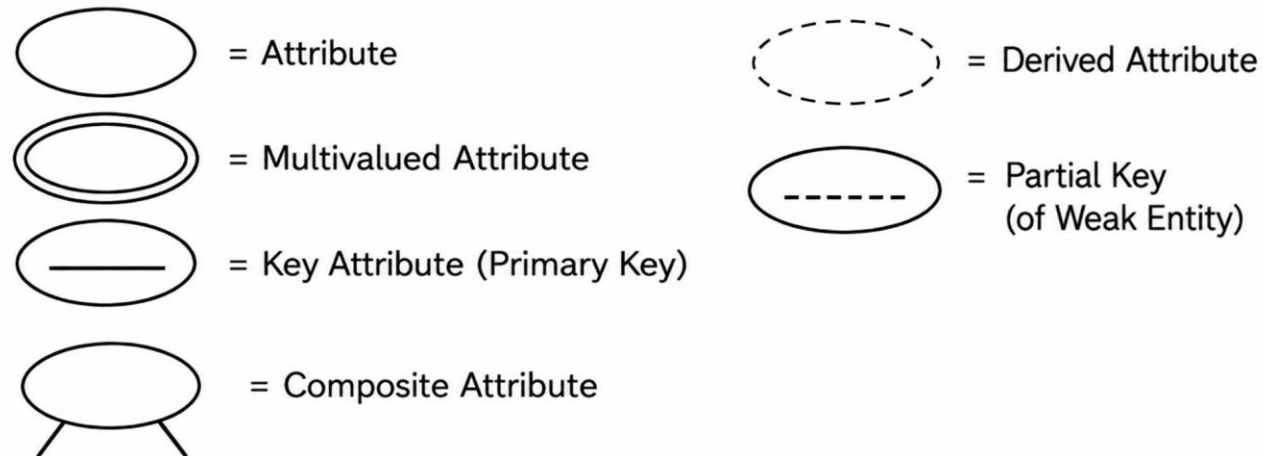
- Lucidchart (<https://lucid.co/lucidchart>, lucid.app)
- Use UML Entity Relations Diagram Template
- **Activity**: Draw the two entites Employee and Department



Attributes

An **attribute** describes a characteristic of an entity or a property of a relationship. For example, the EMPLOYEE entity might have:

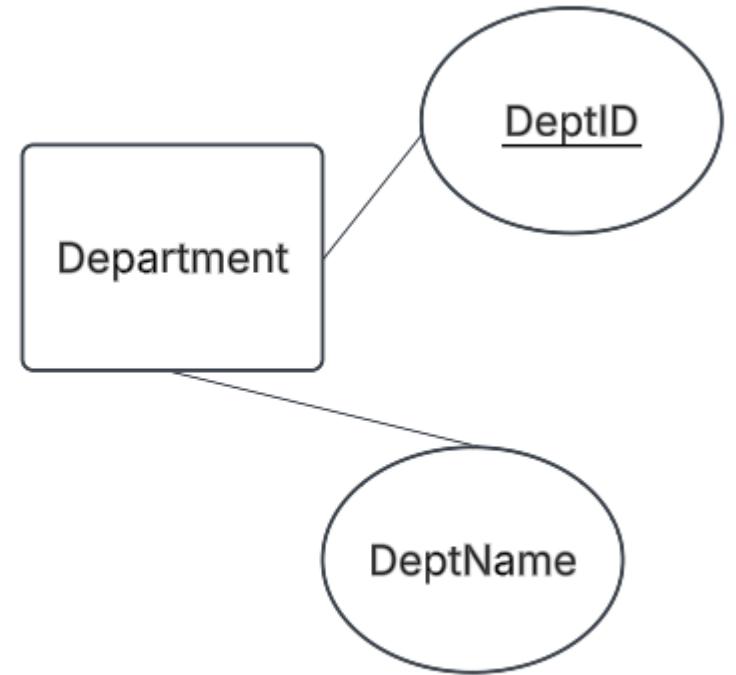
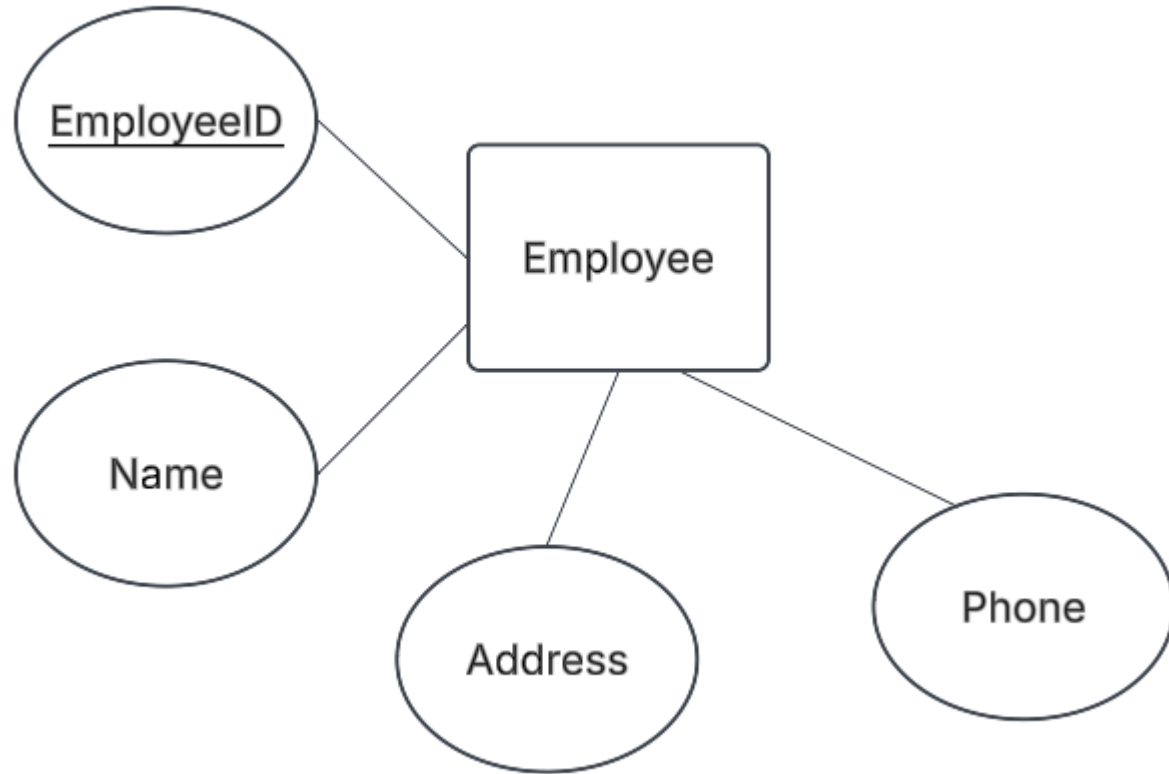
- EmployeeID
- Name
- Phone
- Address



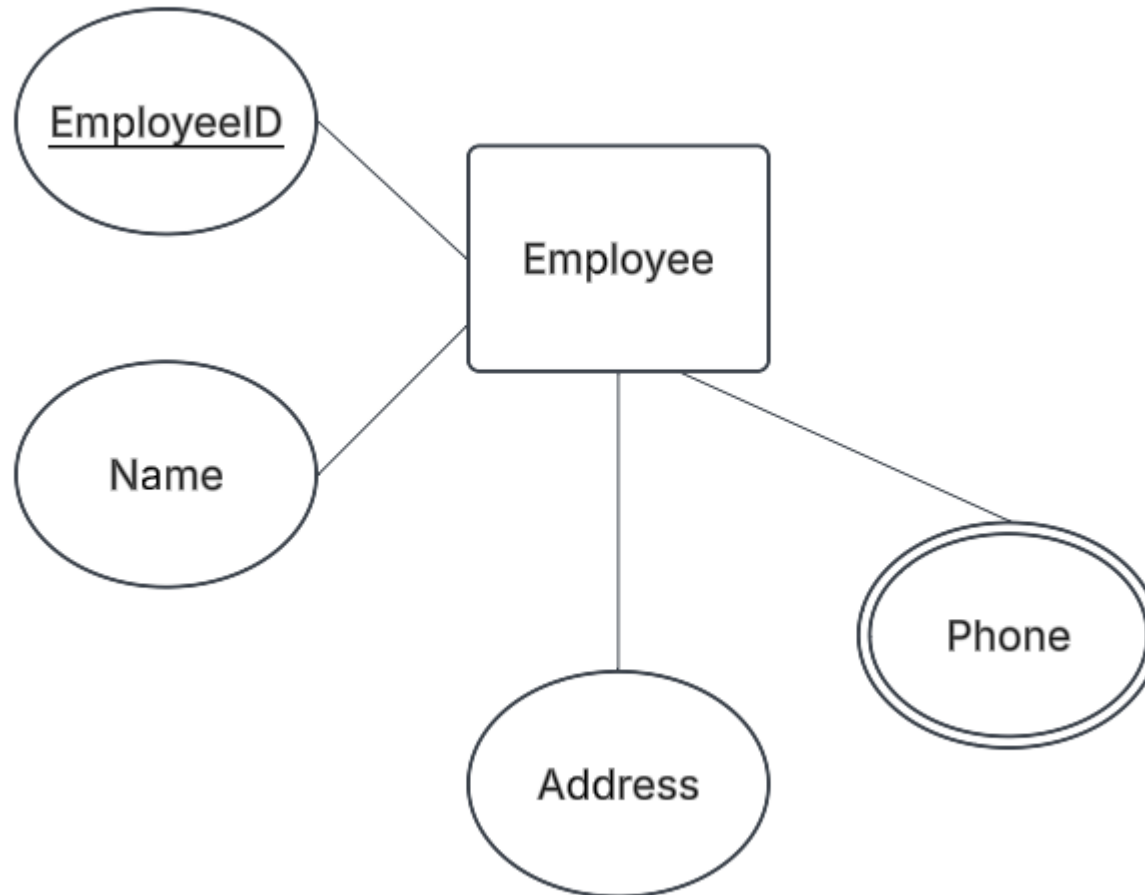
In Chen notation, attributes are represented by **ovals** connected to their entity.

Different types of attributes can also be represented, such as **key attributes**, **composite attributes**, **multivalued attributes**, and **derived attributes**.

Activity: Draw the Appropriate Attributes for Employee and Department Entites

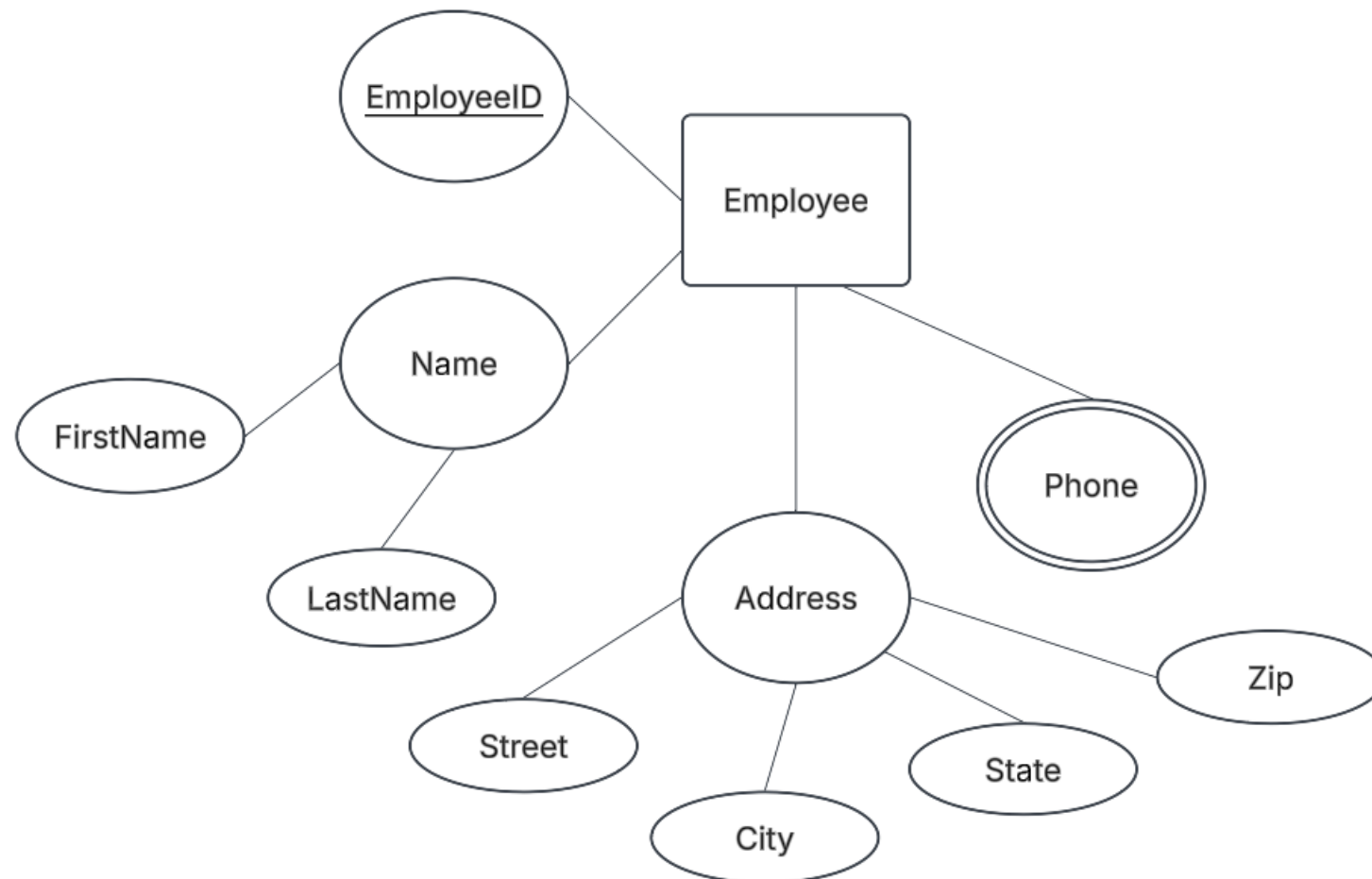


Activity: Draw the correct attribute symbol. The employee can have more than one phone record in the company

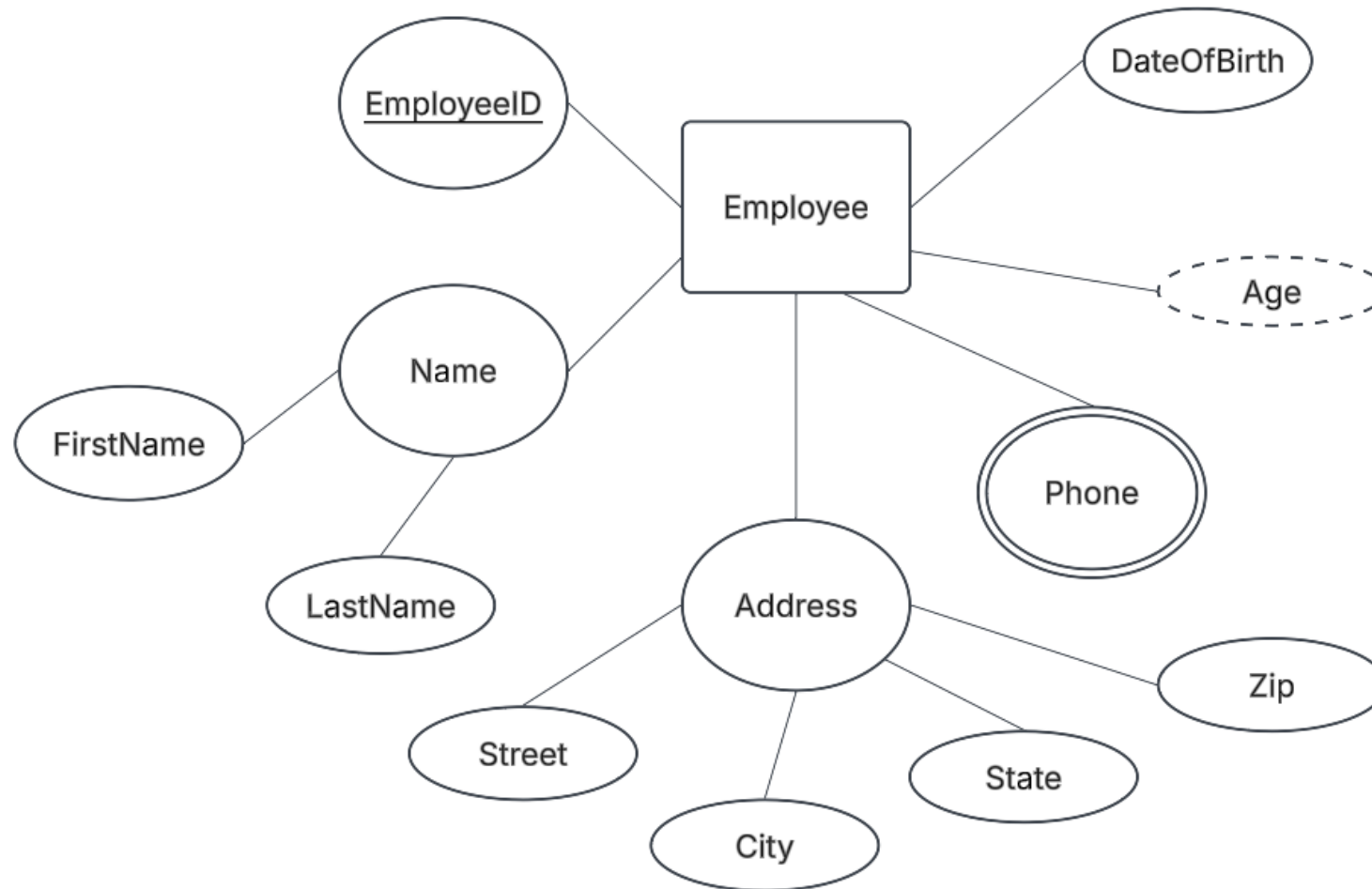


Activity:

1. The company wants the ability to sort the employees based on their last names and first names.
2. The company wants to be able to determine which city each employee resides.



Activity: Now, what if the company needs to know the age of each employee?



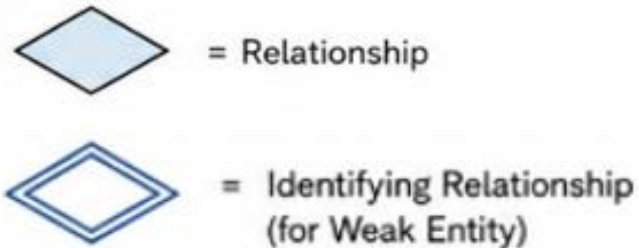
Relationships

A **relationship** describes an association between entities.

For example:

An EMPLOYEE **WORKS_FOR** a DEPARTMENT.

In Chen notation, the relationship is represented by a **diamond**:



For **relationships**, you can have:

- Unary (recursive)
- Binary
- Ternary
- Identifying relationship (for weak entities)

Types of Relationships:

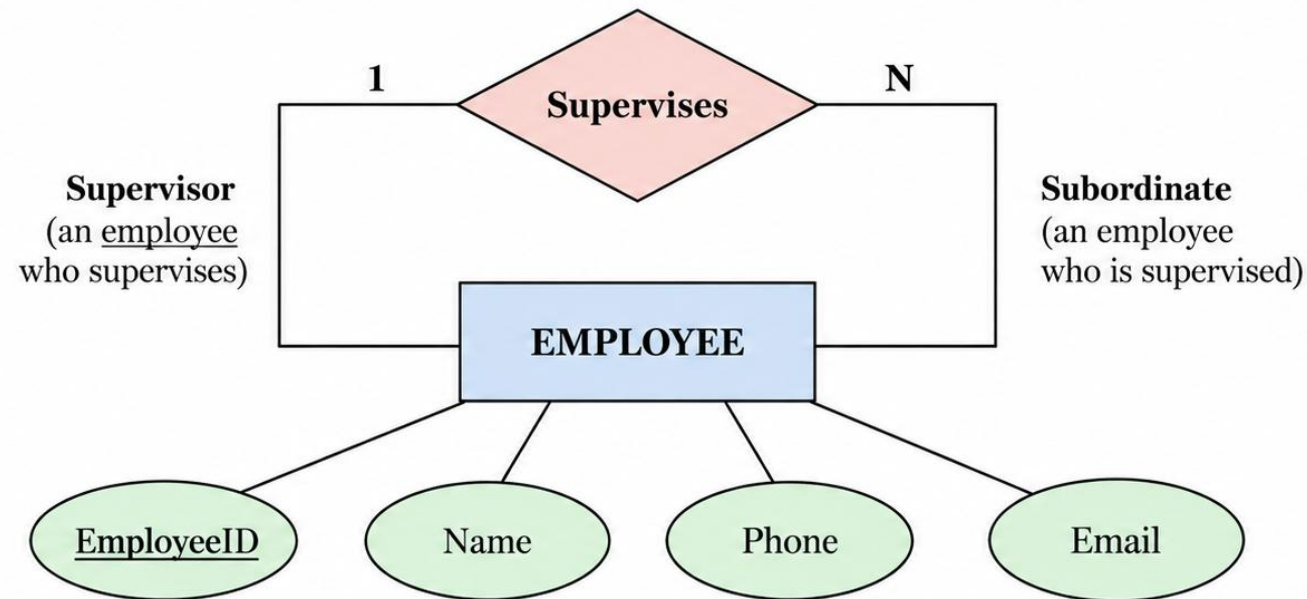
- **Unary (recursive)** — involves **one entity type**. Example: EMPLOYEE — Supervises — EMPLOYEE
- **Binary** — involves **two entity types**. Example: EMPLOYEE — WorksFor — DEPARTMENT
- **Ternary** — involves **three entity types simultaneously**. Example: SUPPLIER — Supplies — PART — PROJECT
- **N-ary** – involves more than three entity types simultaneously.

In terms of **relationship degree**, the **binary relationship is by far the most common** in ERDs.

Activity: An employee can supervise many employees, and each employee is supervised by at most one employee.

Unary (Recursive) Relationship

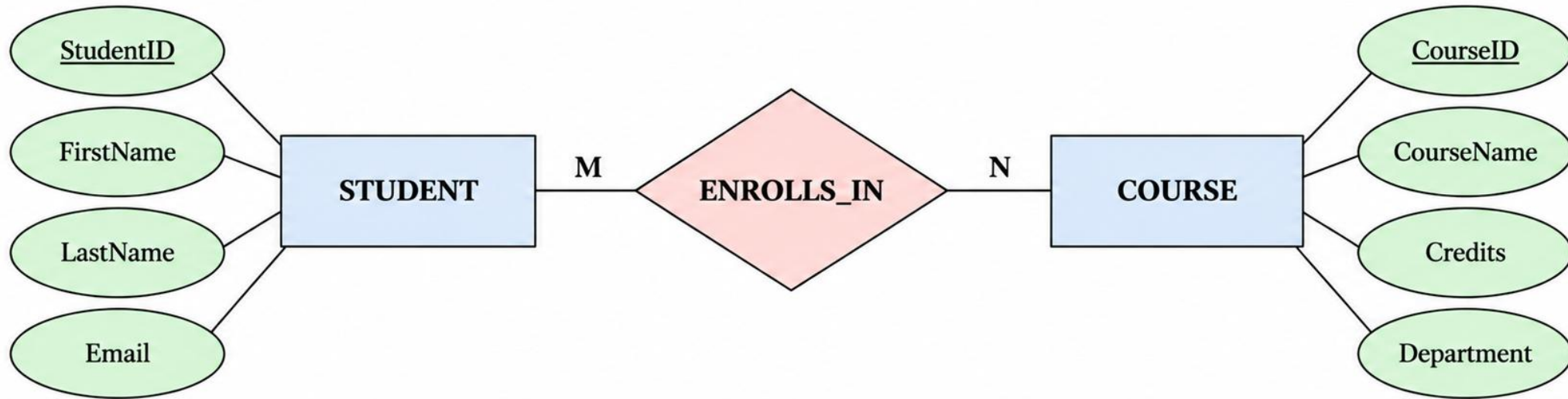
A unary relationship involves one entity type that is related to itself.



Example: An employee can supervise many employees, and each employee is supervised by at most one employee (or none).

Binary Relationship

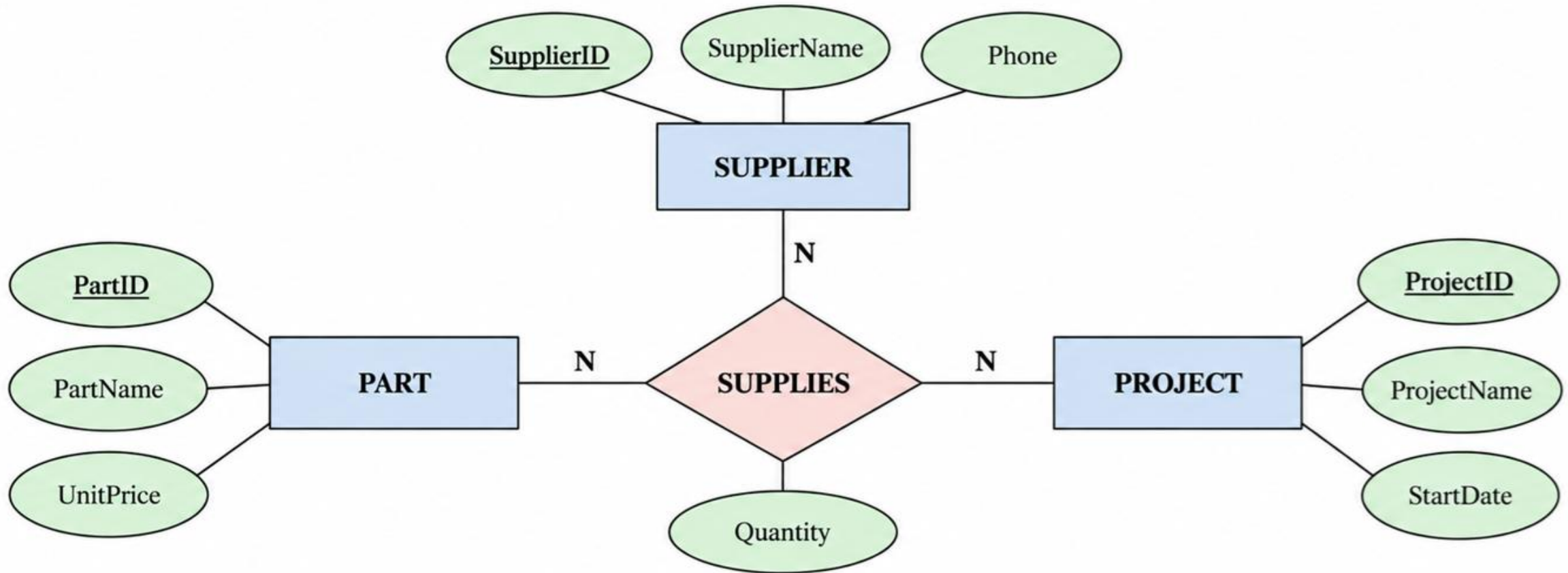
A binary relationship involves two entity types.



Example: A student can enroll in many courses, and a course can have many students.
This is a binary (many-to-many) relationship between STUDENT and COURSE.
(Cardinality shown as M:N)

Ternary Relationship

A ternary relationship involves three entity types simultaneously.



Example: A supplier supplies a part to a project, and we keep track of the quantity supplied. The relationship involves SUPPLIER, PART, and PROJECT simultaneously.

Why Use a Ternary Relationship?

A ternary relationship involves three entity types that must be considered together to represent a complete fact.

Why not use separate binary relationships?

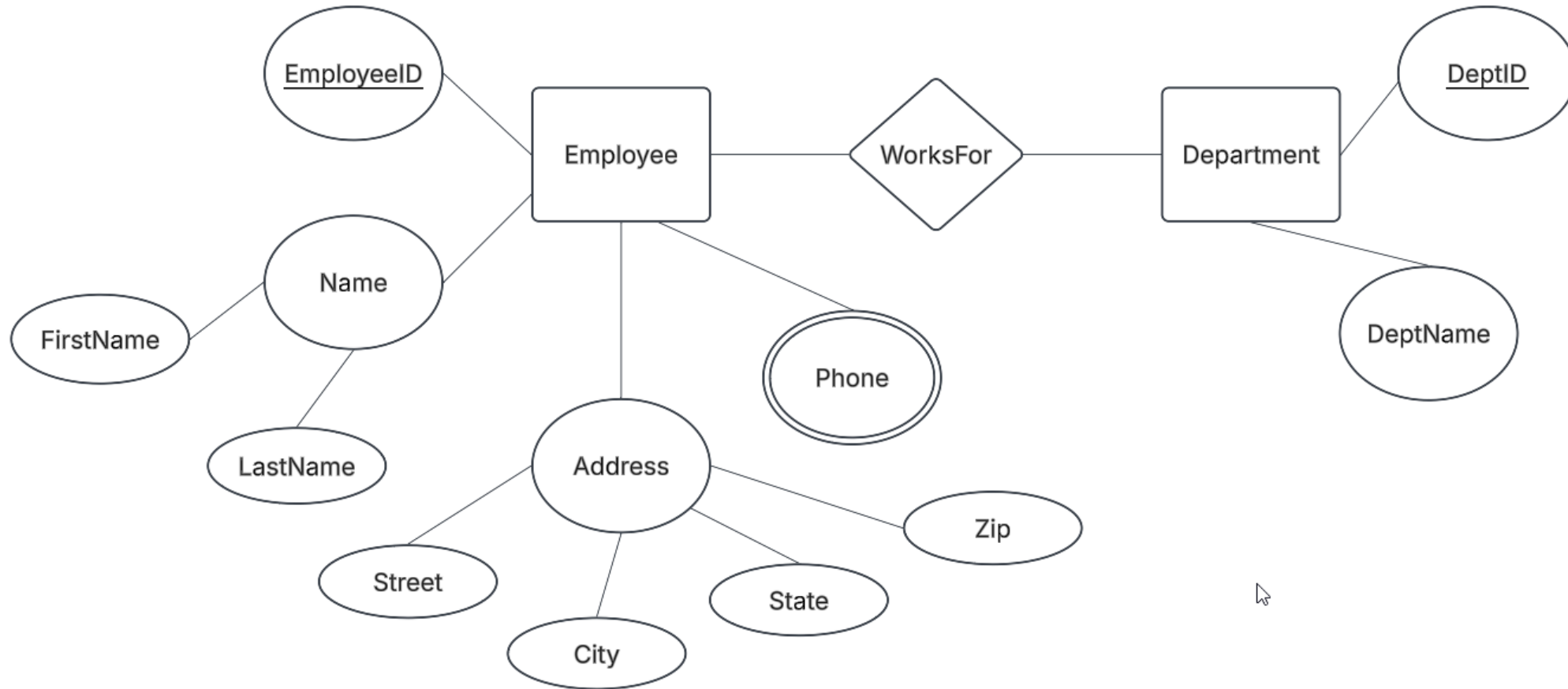
- Supplier — Part tells us **what** the supplier supplies.
- Supplier — Project tells us **which projects** the supplier serves.
- Part — Project tells us **which parts** a project uses.
- But these separate relationships **do not tell us which supplier supplies which specific part to which specific project.**

Use a ternary relationship when breaking it into binary relationships would lose the meaning of the three-way association.

A very useful test is:

If I break this into binary relationships, can I still determine exactly which A is associated with which B and which C?

Activity: Draw the relations between Employee and Department using a WorksFor relationship



- **Weak Entity**

A **weak entity** is an entity that **cannot be uniquely identified by its own attributes alone**. It depends on another entity, called the **owner (or strong) entity**, for its identification.

In Chen notation, a weak entity is shown with a **double rectangle**.

- *Example:* A **Dependent** may be identified by DependentName, but that name may not be unique across all employees. The Dependent is identified using the employee's key together with its own partial key.

- **Partial Key**

A **partial key** is an attribute of a weak entity that distinguishes weak-entity instances belonging to the **same owner entity**.

It is typically shown with a **dashed underline**.

- *Example:* DependentName can be the partial key of Dependent.

- **Identifying Relationship**

An **identifying relationship** connects a weak entity to the strong entity that owns and identifies it.

In Chen notation, it is shown with a **double diamond**.

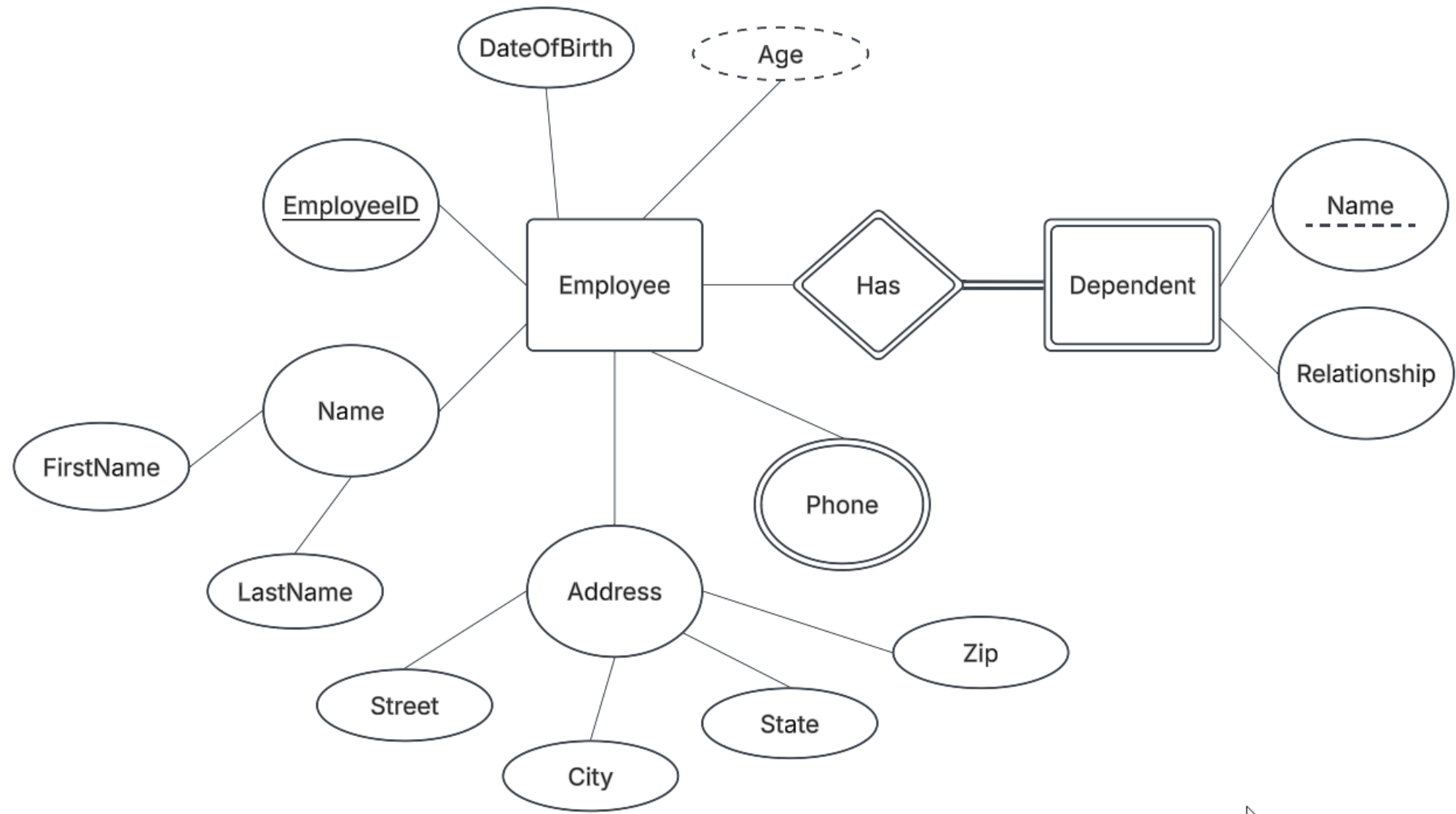
- *Example:*

Employee — Has — Dependent

- Here, Has is the identifying relationship.

- **Total Participation of the Weak Entity**

A weak entity must have **total participation** in its identifying relationship because it cannot exist without its owner entity. This is shown using a **double line** between the weak entity and the identifying relationship.



Cardinality

An ERD can also specify **cardinality**, which describes how many instances of one entity can be associated with instances of another entity.

For example:

EMPLOYEE — N — ◇ WORKS_FOR ◇ — 1 — DEPARTMENT

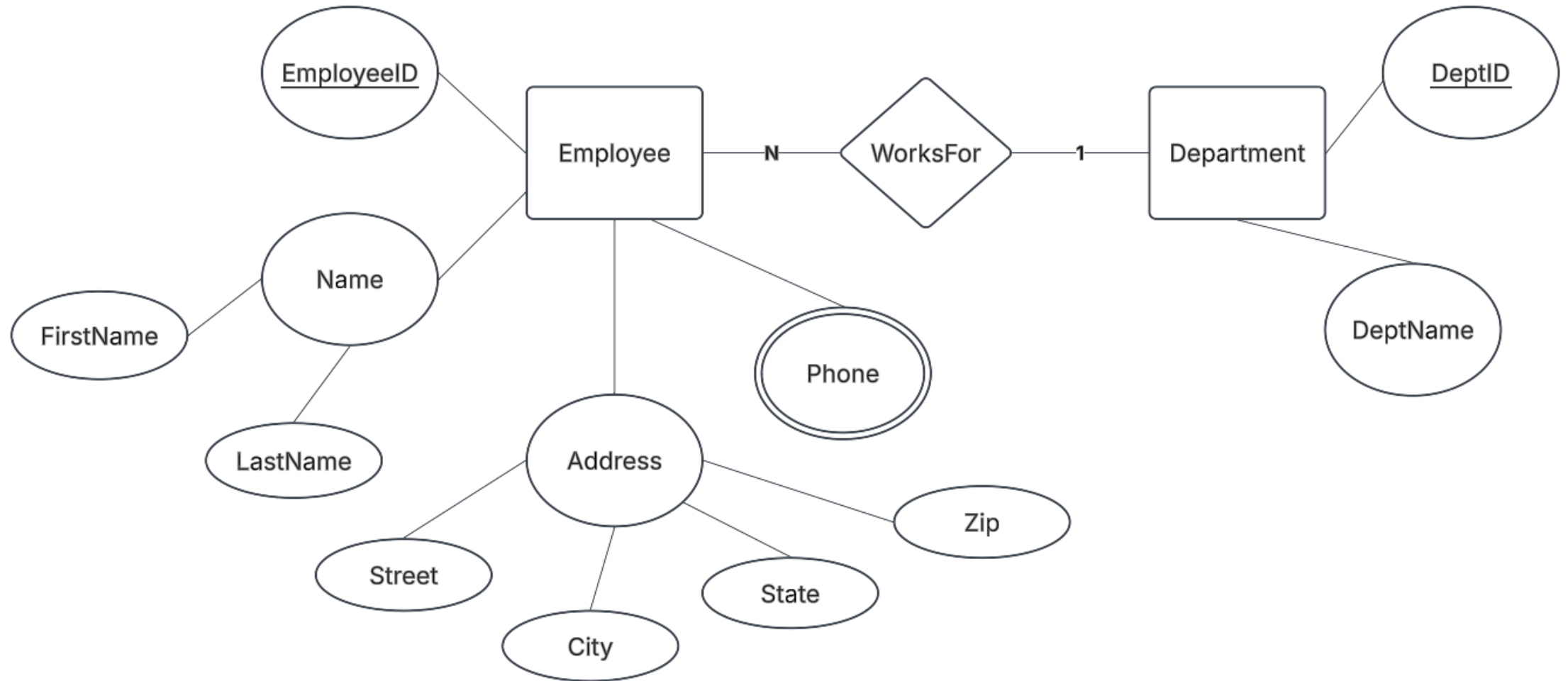
This represents a **many-to-one relationship**:

Many employees can work for one department.

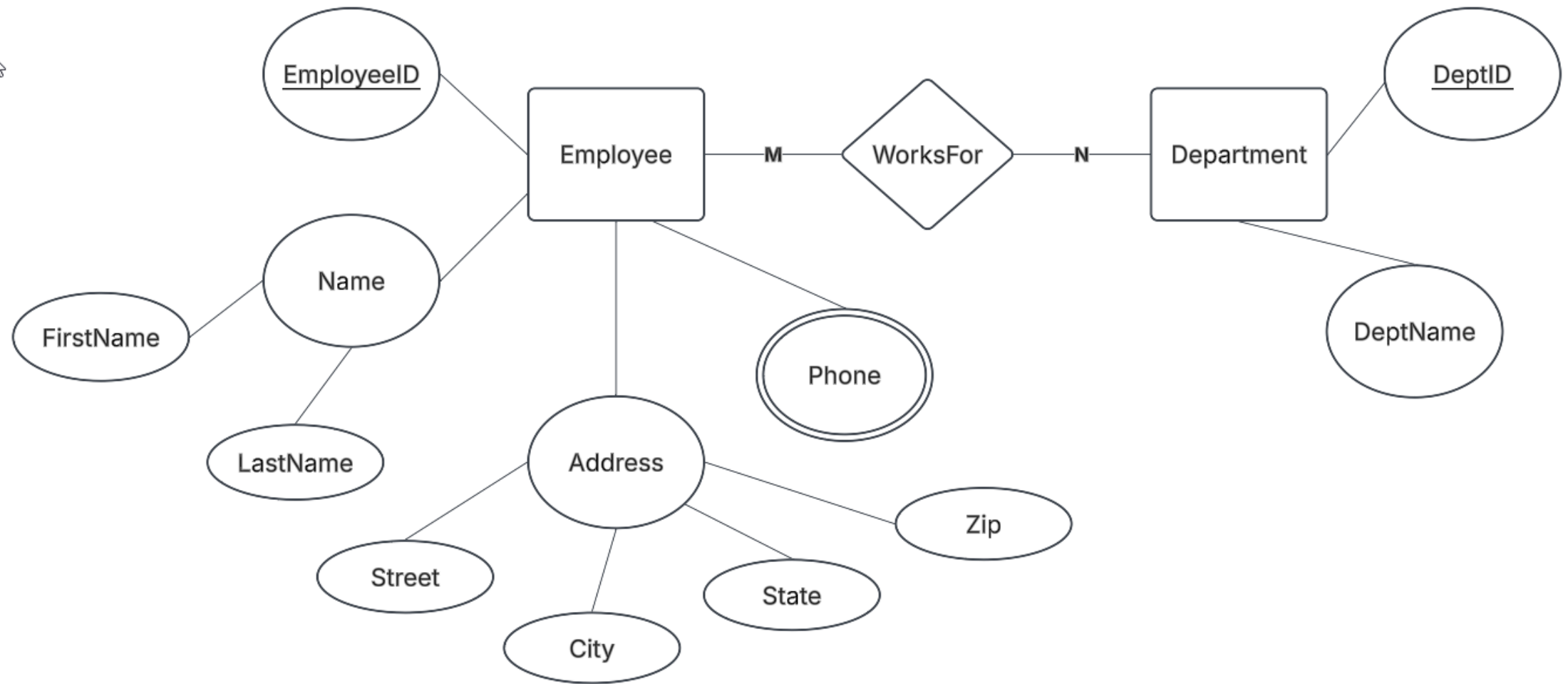
Common cardinalities include:

- **1:1** — One-to-One
- **1:N** — One-to-Many
- **M:N** — Many-to-Many

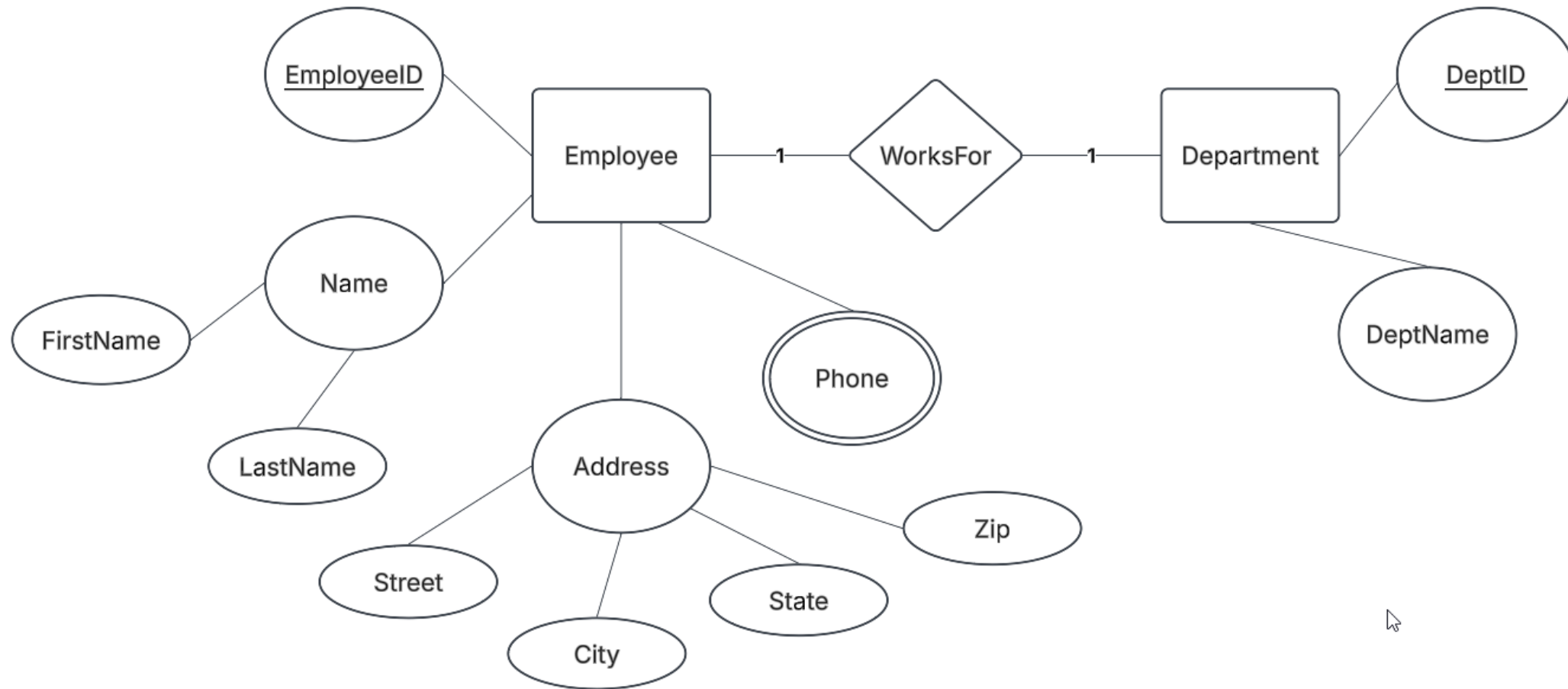
Activity: Add the cardinality. An employee can work for at most one department



Activity: An employee can work for more than one department



Activity: An employee can work for at most one department.
A department can have at most one employee.



What can you say about this ERD, if it's a 1-1 relationship?

Participation

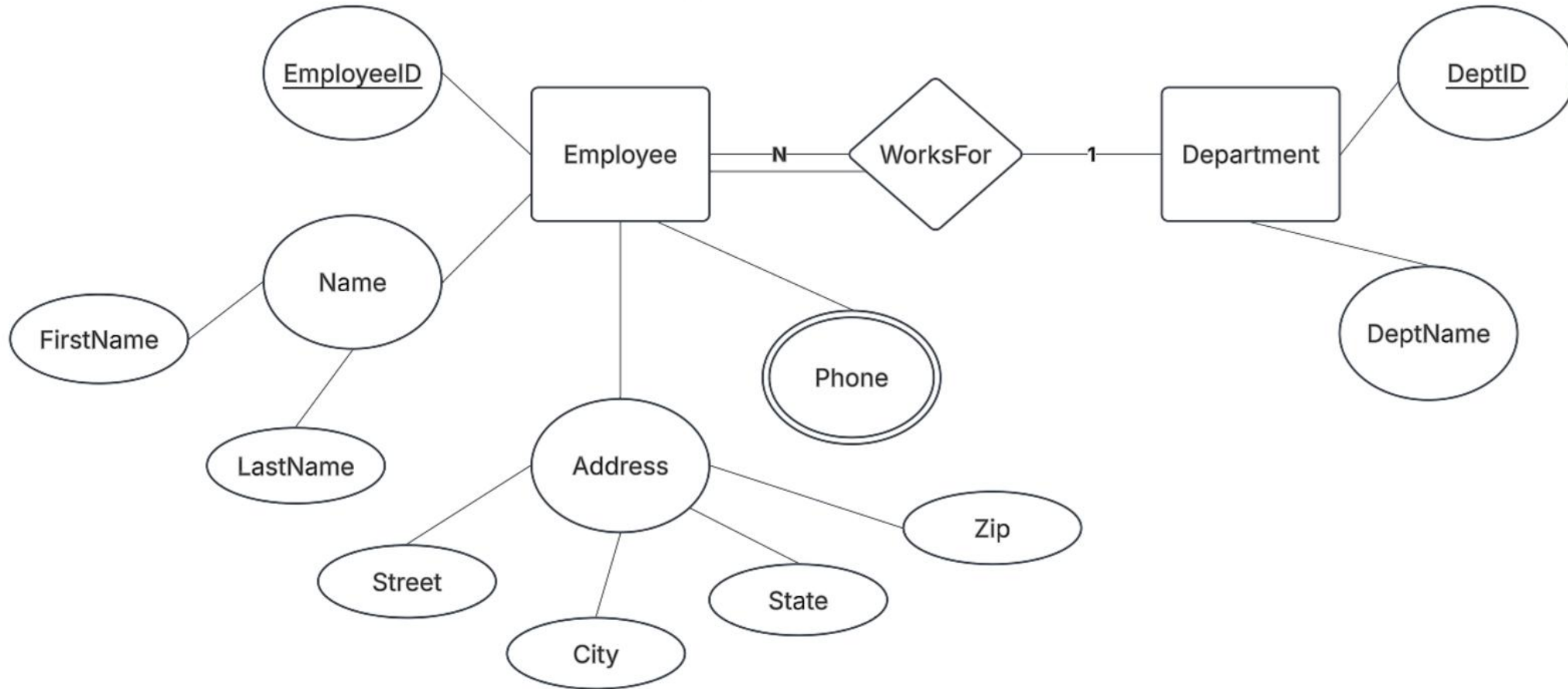
ERDs can also indicate whether participation in a relationship is **required or optional**.

In traditional Chen notation:

- **Single line** → Partial participation
- **Double line** → Total participation

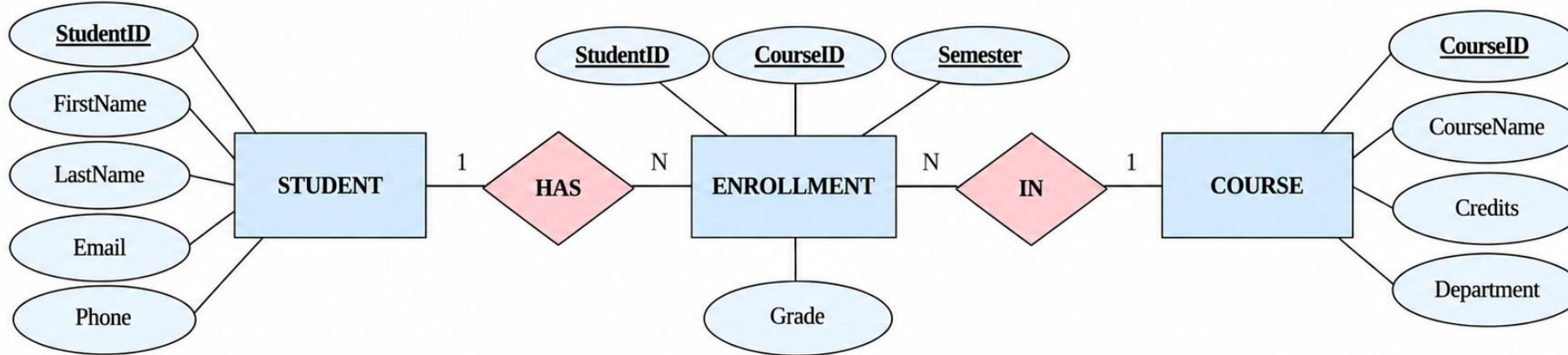
Activity:

Business Constraint: Every employee must belong to a department



If the business rules do not state that participation is mandatory, partial participation is often assumed.

Chen Notation ERD: Student –Enrollment –Course (Three Entities with Composite Key)



Legend (Chen Notation)

- = Entity
- = Relationship
- = Attribute
- = Primary Key (underlined)
- = Partial Key (dashed underline)
- 1, N** = Cardinality (1 = one, N = many)

Notes

1. Student, Enrollment, and Course are three separate entities.
2. A student can have many enrollments (1 : N).
3. A course can have many enrollments (1 : N).
4. Each enrollment is associated with exactly one student and exactly one course.
5. The primary key of ENROLLMENT is a composite key (StudentID, CourseID, Semester) to uniquely identify a student's enrollment in a course in a specific semester.
6. Grade is an attribute of ENROLLMENT (not part of the key).

Why ERDs Are Important

An ERD allows a database designer to plan the structure of a database **before creating the actual tables**. It can help identify missing data, unnecessary duplication, incorrect relationships, and important business rules early in the design process.

Once the ERD has been completed, it can be converted into a **relational schema**, which can then be implemented as tables using SQL.